LINGUAGEM DE PROGRAMAÇÃO I

Exercícios de Aprendizagem

Carlos Henrique Lima

SI-241

2025

1 - Explique o processo híbrido de tradução de código-fonte feito pelo Java, evidenciando a fase de complilação, interpretação e o papel da JRE neste processo.

R - O Java é híbrido porque primeiro compila o código-fonte em bytecode e depois interpreta e otimiza esse bytecode em tempo de execução, usando a JVM dentro da JRE.

2 - Qual a principal vantagem do Java em relação a linguagens puramente compiladas? E a(s) desvantagem(ns)?

R – A principal vantagem é que o Java gera bytecode, que pode ser executado em qualquer sistema que tenha a JVM, ou seja, o mesmo programa roda em Windows, Linux, macOS, etc. E as desvantagens são o desempenho menor do que linguagens puramente compiladas, e o maior uso de memória.

3 - Explique como compilar e executar um programa em Java via linha de comando?

R - Para compilar e executar um programa Java via linha de comando, basta escrever o código em um arquivo ".java", compilar com o comando "javac NomeDoArquivo.java", e depois executar com "java NomeDaClasse".

4 - Defina o que são classes e objetos em Java. Explique a diferença entre eles e exemplifique com uma situação do mundo real.

R - Em Java, classes são modelos ou moldes que definem as características e comportamentos de algo, enquanto objetos são as instâncias concretas criadas a partir dessas classes. A classe é como um plano ou projeto, e o objeto é o produto real construído com base nesse plano. A classe Carro pode definir atributos como cor, marca e velocidade, e métodos como acelerar() e frear(), já um objeto dessa classe seria um carro específico, como um carro azul da marca Toyota.

5 - Em Programação Orientada a Objetos (POO), o recurso que permite a uma classe controlar o acesso a seus atributos e métodos, escondendo detalhes internos e expondo apenas o necessário, é chamado de encapsulamento.

Em Java, qual é o papel dos modificadores de visibilidade public e

private? Dê exemplos práticos de como eles contribuem para o

encapsulamento.

R - Em Java, os modificadores de visibilidade “public” e “private” definem quem pode acessar os atributos e métodos de uma classe, sendo fundamentais para o encapsulamento. O “private” restringe o acesso, permitindo que o atributo ou método seja usado apenas dentro da própria classe. Já o “public” permite que ele seja acessado por qualquer outra classe.

```
public class exercicio5 {

    private double saldo;

    public void depositar(double valor) {
        saldo += valor;
    }

    public double verificarSaldo() {
        return saldo;
    }

}
```

Nesse exemplo, o atributo saldo é “private”, então não pode ser alterado diretamente de fora da classe. O acesso e a modificação só acontecem por meio dos métodos públicos “depositar()” e “verificarSaldo()”, garantindo o encapsulamento.

6 - Em Java, quando os atributos de uma classe são declarados como

private, eles não podem ser acessados diretamente por outras classes.

Para permitir o acesso e a modificação desses valores de forma controlada,

é necessário criar métodos específicos.

Crie uma classe chamada Produto com os atributos privados nome

(String) e preco (double). Depois, implemente métodos que permitam:

− definir o nome de um produto setNome;

− definir o preçp de um produto setPreco;

− consultar o nome (getNome) e o preço (getPreco) já armazenados.

Por fim, crie um objeto da classe Produto e utilize esses métodos para alterar e exibir os valores.

R –

```
public class Main {

    public static void main(String[] args) {

        Produto p = new Produto();

        p.setNome("Notebook");
        p.setPreco(3500.00);

        System.out.println("Produto: " + p.getNome());
        System.out.println("Preço: R$ " + p.getPreco());
    }
}

public class Produto {

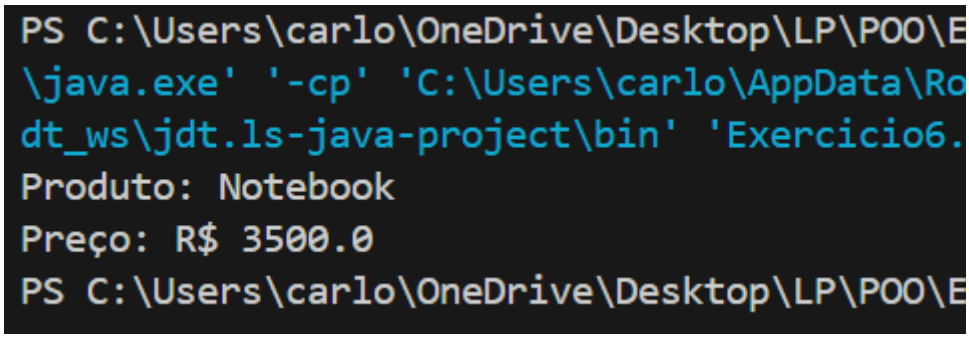
    private String nome;
    private double preco;

    // Método para definir o nome
    public void setNome(String nome) {
        this.nome = nome;
    }
```

```
    // Método para definir o preço
    public void setPreco(double preco) {
        this.preco = preco;
    }

    // Método para consultar o nome
    public String getNome() {
        return nome;
    }

    // Método para consultar o preço
    public double getPreco() {
        return preco;
    }
}
```



7 - Crie uma classe chamada Aluno com os atributos privados nome (String) e três notas (nota1, nota2, nota3, do tipo double).

Implemente métodos que permitam:

− armazenar as notas dos alunos;

− calcular a média das notas;

− verificar, usando uma estrutura de decisão, se o aluno está aprovado (média maior ou igual a 60) ou reprovado.

No método principal, crie um objeto da classe Aluno, cadastre suas notas e exiba a média e a situação final.

R –

```
public class Main {

    public static void main(String[] args) {
        Aluno aluno = new Aluno();

        aluno.setNome("Carlos");
        aluno.setNotas(70, 85, 50);

        double media = aluno.calcularMedia();
        String situacao = aluno.verificarSituacao();

        System.out.println("Aluno: " + aluno.nomealuno);
        System.out.println("Média: " + media);
        System.out.println("Situação: " + situacao);
    }
}

public class Aluno {

    String nomealuno;
    private double nota1;
    private double nota2;
    private double nota3;

    public void setNome(String nomealuno) {
        this.nomealuno = nomealuno;
```

```
    }

    public void setNotas(double n1, double n2, double n3) {
        this.nota1 = n1;
        this.nota2 = n2;
        this.nota3 = n3;
    }

    // Método para calcular a média
    public double calcularMedia() {
        return (nota1 + nota2 + nota3) / 3;
    }

    // Método para verificar a situação do aluno
    public String verificarSituacao() {
        double media = calcularMedia();
        if (media >= 60) {
            return "Aprovado";
        } else {
            return "Reprovado";
        }
    }
}
```

```
PS C:\Users\carlo\OneDrive\Desktop\Exercicio7>  & 'C
AppData\Roaming\Code\User\workspaceStorage\22b6c4b98
ain'
Aluno: Carlos
Média: 68.33333333333333
Situação: Aprovado
PS C:\Users\carlo\OneDrive\Desktop\Exercicio7>
```

8 -

Crie uma classe chamada `Ponto` com os atributos privados `x` e `y` (ambos do tipo `double`), representando um ponto no plano cartesiano.
Implemente métodos que permitam:

- definir e acessar os valores de `x` e `y`;
- calcular a distância entre dois pontos, utilizando a fórmula da distância entre dois pontos no plano:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- exibir as coordenadas de um ponto.

No método principal, crie dois objetos da classe `Ponto`, atribua valores às suas coordenadas, exiba estas coordenadas e a distância entre eles.

R –

```
public class Main {

    public static void main(String[] args) {

        Ponto p1 = new Ponto();
        Ponto p2 = new Ponto();

        p1.setX(2);
        p1.setY(3);

        p2.setX(7);
        p2.setY(9);

        System.out.print("Ponto 1: ");
        p1.exibirCoordenadas();

        System.out.print("Ponto 2: ");
        p2.exibirCoordenadas();
```

```
        // Calculando e exibindo a distância
        double distancia = p1.calcularDistancia(p2);
        System.out.println("Distância entre os pontos: " +
distancia);
    }
}

public class Ponto {

    private double x;
    private double y;

    public void setX(double x) {
        this.x = x;
    }

    public void setY(double y) {
        this.y = y;
    }

    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    // Método para exibir as coordenadas do ponto
```

```
    public void exibirCoordenadas() {
        System.out.println("(" + x + ", " + y + ")");
    }

    // Método para calcular a distância entre dois pontos
    public double calcularDistancia(Ponto outro) {
        double dx = outro.x - this.x;
        double dy = outro.y - this.y;
        return Math.sqrt((dx * dx) + (dy * dy));
    }
}
```

```
AppData\Roaming\Code\User\workspaceStorage\8b68867320d1e1
ain'
Ponto 1: (2.0, 3.0)
Ponto 2: (7.0, 9.0)
Distância entre os pontos: 7.810249675906654
```